Assignment 2

Xander Jhaveri

1. Slab Space Saving

Final Answer: 2.831 GB

I assumed that pages were 4096 bytes

To calculate the space saved by using the slab allocator as opposed to using a minimum of one page per object, we must calculate both the number of pages used by the slab allocator, and the number of pages that we would have used with normal paging.

To calculate the number of pages used by the slab allocator, we simply multiply the number of objects by the size of each object, divide by the page size, and round up. We then sum the corresponding values for every object.

$$\text{Pages Allocated with Slab Allocator} = \sum \left\lceil \frac{\text{Object Size} \cdot \text{Number of Objects}}{\text{Page Size}} \right\rceil$$

To calculate the number of pages that would have been allocated when paging per object, we multiply the number of objects by the number of pages per object. We then sum the corresponding values for every object.

$$\begin{aligned}\text{Pages Allocated with Per Object Paging} &= \sum (\text{Pages/Object} \cdot \text{Number of Objects}) \\ &= \sum \left(\left\lceil \frac{\text{Object Size}}{\text{Page Size}} \right\rceil \cdot \text{Number of Objects} \right)\end{aligned}$$

We can then take the difference and multiply by the page size to find the total space saved:

$$\text{Total Space Saved} = (\text{Pages w/ Per Object Paging} - \text{Pages w/ Slab Allocator}) \cdot \text{Page Size}$$

```
#!/usr/bin/awk -f

# Number of Objects: $3
# Object Size: $4

BEGIN {
  PAGE_SIZE = 4096;
}

function ceil(x) {
    return (x == int(x)) ? x : int(x) + (x > 0 ? 1 : 0)
}

{
  current_slab_alloc_pages = ceil(($3 * $4) / PAGE_SIZE)
  current_perobj_pages = ceil($4 / PAGE_SIZE) * $3

  slab_alloc_pages += current_slab_alloc_pages
  perobj_pages += current_perobj_pages
}

END {
  pages_saved = perobj_pages - slab_alloc_pages
  space_saved = pages_saved * PAGE_SIZE

  printf("Pages used with Slab Allocator: %d\n", slab_alloc_pages)
  printf("Pages used with Per Object Paging: %d\n", perobj_pages)
  printf("Pages Saved with Slab Allocator: %d\n", pages_saved)
  printf("Page Size (in bytes): %d\n", PAGE_SIZE)
  printf("Space Saved (in bytes): %d\n", space_saved)
}
```

Listing 1: q1.awk

After putting the given `slabinfo` into a file called `q1slabinfo.txt`, I used the following command to compute the output: `awk -f q1.awk q1slabinfo.txt`. Here is the output generated by the command:

```
Pages used with Slab Allocator: 82497
Pages used with Per Object Paging: 773650
Pages Saved with Slab Allocator: 691153
Page Size (in bytes): 4096
Space Saved (in bytes): 2830962688
```

Listing 2: Question 1. awk Script Output

The amount of space saved using slabs was **2830962688 bytes**, or **2.831 GB**.

2. Buddy Algorithm in C

To implement the buddy algorithm in C, I prompted *Claude* with the following prompt:

```
I want you to implement the most basic possible simulation of the
buddy algorithm, it doesn't even need to stored data, just simulate the free
lists on allocate and free. I want an array of free lists and a simulated
heap size of 256. I want an allocate(size) method to return a uint_8 that
serves as the address, and a free(addr) to free the memory on the lists.
Prioritize simplicity, no comments or testing.
```

I requested no comments or testing, so that I could introduce print statements and testing myself. This is what was generated:

The code has two main methods: `balloc` and `free`, as well as one important data structure, the array of free lists.

- The array of free lists stores one free list for every 'level' of the buddy allocation.
- `balloc` takes a size as a parameter and finds the optimal level for the allocation to be placed. It then checks if there is space available at the optimal level, and if there is none, it checks one level higher for space. It continues checking until there are no more levels to be checked. Whenever it finds an open block, it will split the block down until it is an optimal size, and then it will mark it as full. It then returns an address as a 1 byte integer.
- `bfree` takes an address, and searches all of the levels for a block with that address to mark that block as free. Once it finds and marks a block as free, it will check the block's buddy (the block at `address XOR 2^order`), and if it is free, it will merge them. It will then check the merged block's buddy to merge and so on.

After this code was generated, I went through and added several print statements for the following events:

- Whenever memory is allocated
- Whenever memory is freed
- Whenever blocks are merged
- Whenever blocks are merged
- Whenever I call `print_flists`, which will print all of the available blocks

```c
#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>

#define HEAP_SIZE 256
#define MAX_ORDER 8
#define LEVELS 9

typedef struct Node {
  uint8_t addr;
  struct Node *next;
} Node;

Node *free_lists[LEVELS];
uint8_t alloc_order[HEAP_SIZE];

void fl_push(int order, uint8_t addr) {
  Node *n = malloc(sizeof(Node));
  n->addr = addr;
  n->next = free_lists[order];
  free_lists[order] = n;
}

uint8_t fl_pop(int order) {
  Node *n = free_lists[order];
  uint8_t addr = n->addr;
  free_lists[order] = n->next;
  free(n);
  return addr;
}

bool fl_remove(int order, uint8_t addr) {
  Node **cur = &free_lists[order];
  while (*cur) {
    if ((*cur)->addr == addr) {
      Node *tmp = *cur;
      *cur = tmp->next;
      free(tmp);
      return true;
    }
    cur = &(*cur)->next;
  }
  return false;
}

uint8_t balloc(size_t size) {
  int order = 0;
  while ((1 << order) < (int)size)
    order++;

  int found = -1;
  for (int i = order; i < LEVELS; i++) {
    if (free_lists[i]) {
      found = i;
      break;
    }
  }
  if (found == -1) {
    // My Addition
    printf("Error: Not enough space for attempted allocation of size %ld\n",
           size);
    return 0xff;
  }

  uint8_t addr = fl_pop(found);
  while (found > order) {
    int prev = found;
    found--;
    // My Addition
    printf("- Splitting size %d block into size %d blocks\n", 1 << prev,
           1 << found);
    fl_push(found, addr + (1 << found));
  }
  // My Addition
  printf("Allocation of size %ld: Using a block with size %d (addr: %d)\n",
         size, 1 << order, addr);

  alloc_order[addr] = order;
  return addr;
}

void bfree(uint8_t addr) {
  int order = alloc_order[addr];
  while (order < MAX_ORDER) {
    uint8_t buddy = addr ^ (1 << order);
    if (fl_remove(order, buddy))
      break;
    if (buddy < addr)
      addr = buddy;
    order++;
    // My Addition
    printf("- Merging size %d blocks into a size %d block\n", 1 << (order -
1),
           1 << order);
  }
  // My Addition
  printf("Freeing address %d; Block of size %d regained\n", addr, 1 <<
order);
  fl_push(order, addr);
}

// My Addition
void print_flists() {
  printf("---- Available Blocks ----\n");
  for (int i = MAX_ORDER; i >= 0; i--) {
    int free_spaces = 0;
    Node *ptr = free_lists[i];
    while (ptr) {
      free_spaces++;
      ptr = ptr->next;
    }
    if (free_spaces != 0) {
      printf("Size %d: %d\n", 1 << i, free_spaces);
    }
  }
  printf("\n");
}

void init() {
  for (int i = 0; i < LEVELS; i++)
    free_lists[i] = NULL;
  fl_push(MAX_ORDER, 0);
}
```

Listing 3: Final `q2.c` file, pre-testing

Here is a test main function that I wrote and its corresponding output:

```c
int main(void) {
  init();

  print_flists();
  uint8_t a = balloc(46);
  uint8_t b = balloc(15);
  print_flists();
  bfree(a);
  print_flists();
  uint8_t c = balloc(29);
  uint8_t d = balloc(119);
  print_flists();
  bfree(c);
  print_flists();
  uint8_t e = balloc(12);
  print_flists();
  bfree(b);
  bfree(e);
  print_flists();
  uint8_t f = balloc(31);
  print_flists();
  uint8_t g = balloc(79);
  print_flists();
  bfree(d);
  print_flists();
  uint8_t h = balloc(56);
  print_flists();

  return EXIT_SUCCESS;
}
```

Listing 4: Test `main` function

```
---- Available Blocks ----
Size 256: 1

- Splitting size 256 block into size 128 blocks
- Splitting size 128 block into size 64 blocks
Allocation of size 46: Using a block with size 64 (addr: 0)
- Splitting size 64 block into size 32 blocks
- Splitting size 32 block into size 16 blocks
Allocation of size 15: Using a block with size 16 (addr: 64)
---- Available Blocks ----
Size 128: 1
Size 32: 1
Size 16: 1

Freeing address 0; Block of size 64 regained
---- Available Blocks ----
Size 128: 1
Size 64: 1
Size 32: 1
Size 16: 1

Allocation of size 29: Using a block with size 32 (addr: 96)
Allocation of size 119: Using a block with size 128 (addr: 128)
---- Available Blocks ----
Size 64: 1
Size 16: 1

Freeing address 96; Block of size 32 regained
---- Available Blocks ----
Size 64: 1
Size 32: 1
Size 16: 1

Allocation of size 12: Using a block with size 16 (addr: 80)
---- Available Blocks ----
Size 64: 1
Size 32: 1

Freeing address 64; Block of size 16 regained
- Merging size 16 blocks into a size 32 block
- Merging size 32 blocks into a size 64 block
- Merging size 64 blocks into a size 128 block
Freeing address 0; Block of size 128 regained
---- Available Blocks ----
Size 128: 1

- Splitting size 128 block into size 64 blocks
- Splitting size 64 block into size 32 blocks
Allocation of size 31: Using a block with size 32 (addr: 0)
---- Available Blocks ----
Size 64: 1
Size 32: 1

Error: Not enough space for attempted allocation of size 79
---- Available Blocks ----
Size 64: 1
Size 32: 1

Freeing address 128; Block of size 128 regained
---- Available Blocks ----
Size 128: 1
Size 64: 1
Size 32: 1

Allocation of size 56: Using a block with size 64 (addr: 64)
---- Available Blocks ----
Size 128: 1
Size 32: 1
```

Listing 5: Output from `cc q2.c && ./a.out`

3. The Effect of Kernel Parameters

Note: I apologize, I really struggled to get good data for this question. I have had a bunch of conversations with Claude about why the parameters weren't having a significant effect, and I'm still not sure I understand completely. Details below

For this problem, I began by logging into my root machine and running the following command to ensure that the parameters for `swappiness`, `user_reserve_kbytes`, and `vfs_cache_pressure` were set to their defaults:

```
sudo sysctl -w vm.swappiness=10 -w vm.user_reserve_kbytes=63201 -w vm
vfs_cache_pressure=50
```

I then ran 5 `mempigs` in the background, waited 20 seconds, and then stored the `top` data, including the statistics about `Swap`, `Mem`, and `buff/cache`. I did this using the following bash script:

```bash
#!/usr/bin/env bash

./mempig &
mempid1=$!
./mempig &
mempid2=$!
./mempig &
mempid3=$!
./mempig &
mempid4=$!
./mempig &
mempid5=$!

sleep 20

top -b -n 1 > $1
kill "$mempid1"
kill "$mempid2"
kill "$mempid3"
kill "$mempid4"
kill "$mempid5"
```

Listing 6: `generate_top.bash`

Note: I used the same `mempig` C program that you used in you Tuesday Demo 10

I then changed the parameters, and ran the script again, repeating ten times with different parameter combinations each time. Here are the combinations I used:

- "default_params":
  - `vm.swappiness=10`
  - `vm.user_reserve_kbytes=63201`
  - `vfs_cache_pressure=50`
- "swappiness=1":
  - `vm.swappiness=1`
  - `vm.user_reserve_kbytes=63201`
  - `vfs_cache_pressure=50`
- "swappiness=100":
  - `vm.swappiness=100`
  - `vm.user_reserve_kbytes=63201`
  - `vfs_cache_pressure=50`
- "10x_user_reserve":
  - `vm.swappiness=10`
  - `vm.user_reserve_kbytes=632010`
  - `vfs_cache_pressure=50`
- "10xless_user_reserve":
  - `vm.swappiness=10`
  - `vm.user_reserve_kbytes=6320`
  - `vfs_cache_pressure=50`
- "cache_pressure=5":
  - `vm.swappiness=10`
  - `vm.user_reserve_kbytes=63201`
  - `vfs_cache_pressure=5`
- "cache_pressure=500":
  - `vm.swappiness=10`
  - `vm.user_reserve_kbytes=63201`
  - `vfs_cache_pressure=500`
- "10x_everything":
  - `vm.swappiness=100`
  - `vm.user_reserve_kbytes=632010`
  - `vfs_cache_pressure=500`
- "10xless_everything":
  - `vm.swappiness=1`
  - `vm.user_reserve_kbytes=6320`
  - `vfs_cache_pressure=5`
- "swap=1_reserve=10x_pressure=5":
  - `vm.swappiness=1`
  - `vm.user_reserve_kbytes=632010`
  - `vfs_cache_pressure=5`

First I will look at the Swap statistics in top with respect to the kernel parameters. In all of the trials, the load started at approximately 0.3MB swapped out before running the `mempigs`. Here is the data after running the `mempigs`:

Parameter Combination	Total RAM (MiB)	Swap Used (MiB)
10x_everything	4096.0	2469.2
10x_user_reserve	4096.0	2462.7
10xless_everything	4096.0	2460.4
10xless_user_reserve	4096.0	2456.3
cache_pressure=5	4096.0	2475.1
cache_pressure=500	4096.0	2465.1
default_params	4096.0	2460.3
swap=1_reserve=10x_pressure=5	4096.0	2477.7
swappiness=1	4096.0	2462.3
swappiness=100	4096.0	2462.6

Table 1: Swap Usage by Parameter Combination

I didn't make a plot of swap usage; I think the plot would've been pretty boring since the swap usage stayed very consistent throughout the parameter changes.

`vm.swappiness` is a kernel parameter that changes how agressively/frequently the OS moves inactive data onto the disk. You'll notice that all of the parameter combinations are very similar in their swap usage. Initially, I thought that I had made a mistake with how I changed `vm.swappiness` and I spent some time looking into it. However, now I believe that this occurred because the RAM was completely filled, so the OS had no choice but to swap out regardless of the swappiness. Here are some statistics on the RAM being completely utilized to back up this theory:

Parameter Combination	Total RAM (MiB)	RAM Used (MiB)
10x_everything	1971.3	1833.1
10x_user_reserve	1971.3	1837.4
10xless_everything	1971.3	1846.6
10xless_user_reserve	1971.3	1843.8
cache_pressure=5	1971.3	1825.5
cache_pressure=500	1971.3	1838.6
default_params	1971.3	1839.1
swap=1_reserve=10x_pressure=5	1971.3	1825.5
swappiness=1	1971.3	1840.0
swappiness=100	1971.3	1839.7

Table 2: RAM Usage by Parameter Combination

Now let's look at disk cache usage with respect to `vfs_cache_pressure`.

`buff/cache` is a statistic in `top` that shows how much RAM is being used as disk cache, and `vfs_cache_pressure` is a parameter that controls how aggressively/frequently the OS will switch out data in the cache. My hypothesis was the following: a higher `vfs_cache_pressure` value should result in the cache being cleared more often, meaning less disk cache and less `buff/cache`. Here is the data:

Parameter Combination	`vfs_cache_pressure`	`buff/cache`
10x_everything	500	62.7 MB
10x_user_reserve	50	53.3 MB
10xless_everything	5	56.4 MB
10xless_user_reserve	50	56.2 MB
cache_pressure=5	5	67.4 MB
cache_pressure=500	500	58.5 MB
default_params	50	56.4 MB
swap=1_reserve=10x_pressure=5	5	63.4 MB
swappiness=1	50	54.3 MB
swappiness=100	50	52.5 MB

Table 3: `vfs_cache_pressure` vs. `buff/cache`

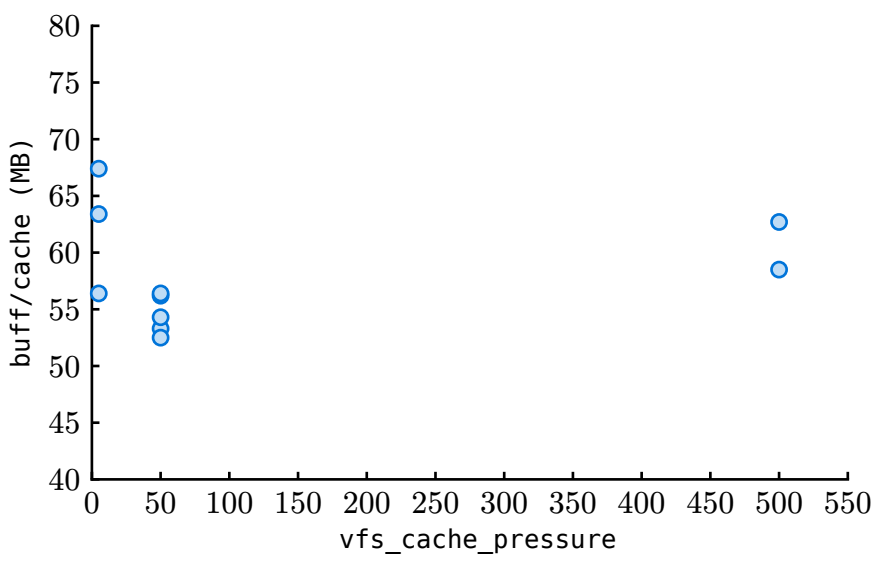

Figure 1: `vfs_cache_pressure` vs. `buff/cache`

Althought this data is relatively inconsistent, I find it interesting because when the `vfs_cache_pressure` is low, there is a lot of memory being used as disk cache. I believe that this is because the cache is being cleared infrequently. However, when `vfs_cache_pressure` is really high, there is also more memory being used as disk cache than a medium `vfs_cache_pressure`. I believe that this is because data is being moved into and out of the cache so quickly, that more RAM is being used as disk cache.

4. The Future of OS Textbooks (ft. Claude)

Summary of my conversation with Claude:

- We discussed how modern OS is focused on the cache
- I learned what asymmetric multiprocessing is
- We discussed how Best/Worst/First/etc. Fit aren't actually used
- Claude agrees that CFS and MLFQ are buffoonery
- Claude inferred that you like venting
- I learned what NUMA is (after I googled it)
- I found out about the importance of having both DRAM and ZRAM
- I learned whta FPGAs are
- I learned that GPU scheduling is far different than CPU scheduling
- I learned that I probably haven't learned enough about security

Me

Hello there! In my OS class, we are using the textbook "Operating Systems: Three Easy Pieces". My professor believes that in the future, OS textbooks will look very different. Do you agree?

Claude

I think your professor has a good point. OS textbooks today focus heavily on processes, threads, memory management, and file systems —concepts that emerged from single-machine computing. As systems evolve (distributed computing, containerization, cloud infrastructure, GPUs becoming first-class citizens), the fundamentals students need to understand will shift.

That said, I'd suspect the core concepts will persist—you'll still need to understand scheduling, synchronization, resource allocation. But yeah, the emphasis and examples will likely look quite different.

What's prompting your professor's view? Is it about specific changes they foresee?

Me

He believes that cache performance and dedicated CPUs along with "asymmetric multiprocessing and power stepping" will be a bigger focus, what is your opinion on this?

Claude

That's a sharp observation. I'd say he's onto something real:

Cache performance —absolutely. Modern systems are already obsessed with this; it'll only get worse (better?). Algorithms that ignore cache locality are increasingly untenable.

Asymmetric multiprocessing + power stepping —yes, this is already happening (ARM's big.LITTLE, for instance). Understanding which cores to pin work to matters more now.

Against pointer-based data structures — here I'm slightly skeptical he means entirely deprecating them. But I get the thrust: if cache misses dominate your runtime, linked lists and trees with scattered pointers become genuinely bad choices compared to cache-friendly arrays, even if their worst-case complexity looks worse on paper. That's a real inversion of what CS textbooks traditionally teach.

So the shift isn't "pointers are evil," it's "worst-case Big-O analysis alone is misleading." Practical performance depends on the hardware you're actually running on.

Does your professor talk about what this means for how you should approach OS problems now?

Me

Kind of. I understand the importance of cache performance and see where dedicated CPUs could be helpful, but I don't know what "asymmetric multiprocessing" or "power stepping" are. Could you explain these?

Claude

Asymmetric multiprocessing: A CPU design where different cores have different capabilities or performance levels. ARM's big.LITTLE is the main example—you have a few high-performance cores and several low-power cores. The OS scheduler picks which core type to run a task on based on its needs.

Power stepping (or dynamic voltage and frequency scaling): The CPU adjusts its clock speed and voltage on the fly based on workload. Under light load, it runs slower and uses less power. Under heavy load, it cranks up. The OS can influence these decisions.

Together they mean: modern CPUs aren't uniform. The scheduler needs to be aware that core A is faster than core B, and that running something on core A burns more power. That's very different from traditional OS textbooks, which assume all cores are identical.

Me

He is also very passionate that the allocation free ft/worst ft/first ft is a terrible way of teaching. He thinks glibc binned lists will replace these ideals

Claude

He's right that the classical allocation strategies are outdated pedagogy. glibc's binned/bucketing approach is what actually matters in practice —it's what real allocators do.

Teaching first/best/worst fit gives students the wrong mental model. They learn to think about fragmentation in abstract terms, when the real problem is: "how do I quickly find a free block of the right size?" Binned lists solve that (O(1) lookup in the right bucket, not O(n) scanning).

That said, I'd argue the concepts underlying those strategies (speed vs. fragmentation tradeoffs) still matter for understanding allocator design. But yeah, leading with first/best/worst fit and then saying "oh but real allocators do something else" is backwards.

Does he advocate teaching it binned-first instead?

Me

Not really. He also thinks these future textbooks will have more on hypervisors and virtual machines vs bare metal machines, with details on emulation and paravirtualization.

Claude

That makes sense. Virtualization is already pervasive (cloud, containers, development environments), yet it gets minimal treatment in most OS courses. Students graduate without understanding hypervisors or the performance implications of paravirtualization vs. full emulation.

It's a practical gap—understanding how a VM actually runs, what traps to the hypervisor, how much overhead you're paying—is more relevant to modern work than some of the bare-metal minutiae.

Does your course actually cover this stuff, or is it still the traditional single-machine OS focus?

Me

Its mostly traditional single-machine focused, but there's a bit on virtualization. He also thinks that there will more of a focus on the types of disks, i.e. holds used only for backup/archive, with seds the norm for persistently attached secondary storage

Claude

That's practical. HDDs as primary storage are already obsolete in most contexts—SSDs are the norm. Teaching disk scheduling algorithms optimized for rotational latency (seek time, etc.) is increasingly irrelevant.

The real OS concerns with SSDs are different: wear leveling, flash translation layers, garbage collection overhead, the performance cliff when the drive fills up. Those are the concepts that matter now.

Though I'd note OSTEP still has value as a foundation—understanding why those old algorithms existed helps you reason about storage generally. But yeah, the emphasis should flip.

Is he planning to update the course based on these ideas, or just venting about textbook gaps?

Me

He just likes venting but also looks forward to a future with more applicable textbooks. Do you think there should be more of a focus on large dram and fast zram with numa and other memory hierarchy heterogeneity?

Claude

Absolutely. NUMA is increasingly relevant (multi-socket servers, AMD's chiplet designs), and understanding memory affinity and cross-socket costs is real knowledge students need.

Large DRAM and zram (compressed memory) are practical realities that traditional "virtual memory with paging" doesn't address well. The hierarchies are getting weirder—you can't just think linearly anymore.

That said, the fundamentals of virtual memory still matter as a foundation. But yeah, teaching modern heterogeneous memory hierarchies and trade-offs (compression overhead vs. bandwidth savings, NUMA penalties) alongside the basics would be more applicable.

Your professor seems to have a coherent vision: move from idealized, simplified models to designs that actually reflect contemporary hardware.

Me

Professor says that the textbook doesn't stress network streaming issues with FPGAs offloading work, and to be honest I don't know what this means, could you explain it and why it could be important for textbooks to include this subject?

Claude

FPGAs are programmable hardware that can offload compute-intensive tasks from the CPU. The "network streaming" part means data flows from the network → FPGA → results back, rather than going CPU → memory → network.

The OS relevance: if you're offloading work to an FPGA, the bottleneck shifts. Traditional OS concerns (context switching, cache coherency) become less important than moving data efficiently to/from the FPGA. You need to understand DMA, interrupt handling for FPGA completion, memory mapping for shared buffers—different OS challenges than CPU-centric systems.

Why it matters for textbooks: FPGAs are increasingly used in datacenters and specialized workloads (HFT, video processing, AI inference). An OS course ignoring them is ignoring real infrastructure students might encounter.

That said, this seems pretty specialized. Does your professor work with FPGAs, or is he just forward-thinking about where systems are headed?

Me

I'm not sure. He also thinks that the textbooks will be more focused on locking issues with massive multithreading. This he has chosen to teach us more about because he believes its important

Claude

That makes sense—traditional OS textbooks cover locks and synchronization, but often with toy examples (a few threads competing). Real systems have hundreds or thousands of threads, which exposes different problems: lock contention, cache line bouncing, priority inversion at scale.

Teaching massive multithreading forces you to think about lock-free algorithms, fine-grained locking, reader-writer locks, condition variables under load—practical concerns that matter in modern server software.

Does he spend significant class time on this, or is it more supplementary material?

Me

Mostly supplementary materials. What about Little's law for cloud provision and automatic scaling, do you think this is explored enough?

Claude

Little's Law (average number in system = arrival rate times average time in system) is powerful for capacity planning and scaling decisions, but I'd guess most OS textbooks don't touch it at all—it's more queueing theory than OS.

That said, it's directly applicable: understanding queue depths, latency under load, and how to dimension resources for auto-scaling requires this kind of thinking. It bridges OS fundamentals with practical cloud operations.

Is your professor covering this in the course?

Me

He is, as the course it as CWRU, he loves Little's Law. He also thinks that just because of the way AI is headed, that AI neural net training and GPU handoff will be more prominent.

Claude

That's forward-looking. GPU scheduling, memory management for large models, and how the OS handles offloading to accelerators are becoming critical. Most OS courses ignore this entirely.

The OS angle is real: how do you fairly share GPUs across multiple training jobs? How does memory coherency work when your model weights live on accelerators? These are becoming practical OS problems, not theoretical ones.

Does he cover GPU scheduling or accelerator management in the course?

Me

We don't cover GPU scheduling (that I know of), but I understand how it could be important for AI. My professor frequently makes fun of MLFQ and CFS for being unfair, impractical, "buffoonery". Do you think there will be more of a real-time priority focus?

Claude

Ha—MLFQ and CFS are designed for general-purpose workloads, not real-time constraints. His point is fair: they're optimized for "fairness" and interactivity in ways that don't match modern needs (latency-sensitive services, SLO guarantees, priority inversion under load).

Real-time scheduling (hard deadlines, priority guarantees) might not fully replace them, but yeah, I'd expect more emphasis on predictable latency and priority-aware scheduling. Cloud services care about meeting SLOs, not just "everyone gets a fair slice."

That said, real-time scheduling has its own complexity—preemption costs, priority inversion, starvation. Not a magic bullet.

Does he advocate for specific scheduling approaches as alternatives?

Me

I don't know if he believes in specific alternatives, but rather he just knows innately that there is a better way. Lastly he thinks that updates, security, user target market mix, and robustness/resilience issues will be covered more; currently there are very few security chapters.

Claude

That's solid. Most OS textbooks treat security as an afterthought (maybe one chapter on access control), but in practice it's woven through everything: privilege escalation, side-channel attacks, supply chain vulnerabilities, update mechanisms.

Updates and resilience are especially neglected—how do you patch a system without downtime? How do systems recover from failures? These matter more than ever, especially in cloud/distributed contexts.

User-targeted design (what users actually need vs. what textbooks teach) is also a fair critique.

Your professor seems to have a coherent philosophy: move from idealized, simplified systems toward practical, real-world concerns. Sounds like a good OS course.

Snapshot Values cause Plane Crashes

According to Claude…

Boeing 737 MAX MCAS Crash (2018 & 2019)

- A faulty sensor tricked the plane's computer into repeatedly pushing the nose down, and the system never bothered checking the other sensors to see if the reading made sense.
- This caused two crashes that killed 346 people.
- The sensor was giving a single bad reading.

B-2 Spirit Crash (2008)

- Rain got into three of the plane's 24 sensors right before takeoff, causing the computer to violently pitch the nose up at the worst possible moment.
- The $1.4 billion plane crashed, even 21 remaining sensors were working perfectly fine.

Boeing 757 Crash (1996)

- Two planes crashed the same year because sensors blocked by a wasp nest fed the autopilot false data, causing it to pull the nose up until the planes stalled.
- In both cases, other sensors were showing completely different numbers, but nothing in the system was designed to notice the difference.